

AER1503 Project Final Report

Optimizing transfer trajectories for Earth-Mars interplanetary orbital maneuvers

Ritik Kothari
Institute for Aerospace Studies
University of Toronto
Toronto, Canada
ritik.kothari@mail.utoronto.ca

Abstract—This paper details the development of an end-to-end Python interplanetary trajectory optimization code, using NASA’s modular OpenMDAO framework. An Earth-Mars one way Hohmann transfer trajectory is considered, and an optimization problem statement is defined. The objective function is originally developed as a Bolza cost functional using variational calculus, under the assumption that perturbative effects such as atmospheric drag, solar radiation pressure and various gravitational forces impact the spacecraft’s mission trajectory. However, due to OpenMDAO’s current limitation of only being able to optimize one objective at a time, the program instead minimizes a simplified total delta-V objective function. With Low Earth Orbit and Low Mars Orbit as design variables, the program finds non-optimal delta-V values to range from 2.280 km/s to 2.306 km/s using standard equations of orbital mechanics. Once the optimizer is run, it is found that optimal delta-V values do not decrease significantly, with reductions ranging from 1.7% to 2.9% based on the chosen initial guess. The slight decrease in reductions supports the belief that Hohmann ellipses are the most energy efficient approaches for travel between circular and coplanar orbits. The OpenMDAO implementation lays the foundation for more sophisticated future optimization studies.

I. INTRODUCTION

With the emergence of NASA’s Artemis program and other global initiatives, the public’s interest in space exploration has been renewed. Given that humanity has already been to the Moon (and will likely get there again), the next feasible target for mankind is Mars. Despite its relatively close proximity to Earth, planning Martian missions is notoriously difficult. Factors such as continually-changing planet positions, spacecraft design constraints, and mission length all contribute to the complexity of this mission type [1].

However, history has shown that both the planning and execution of an unmanned mission to Mars is possible and repeatable. In fact, there have been dozens of Mars mission attempts, the first of which launched in the 20th century [2]. While not all of these attempts were successful, the lessons learned have undoubtedly contributed to what is known about Mars today, including studies on achieving optimal transfers to the planet.

In 1960, German engineer Walter Hohmann showed that the “most efficient way to do [a transfer is] with two impulses, where the initial and final orbits are circular, is to connect opposite sides of the initial and target orbits with an ellipse” [3], [4]. This maneuver, dubbed the “Hohmann Transfer” after

its creator, describes the optimal transfer for interplanetary missions from Earth to Mars.

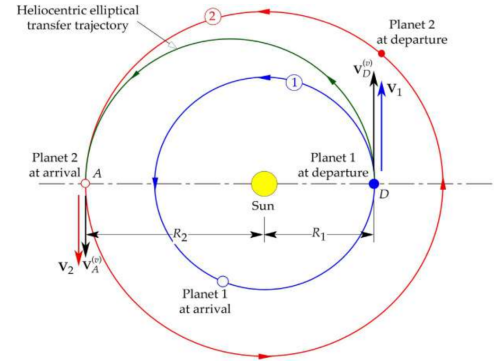


Fig. 1. A Hohmann transfer from inner planet 1 (Earth) to outer planet 2 (Mars). This image is taken from Chapter 8 of Howard Curtis’ *Orbital Mechanics for Engineering Students* [5].

While Hohmann’s solution is elegant in theory and summarizes the problem well, the evolution of technology throughout the 21st century has resulted in the creation of tools and numerical solutions that can optimize interplanetary trajectories even further, given a sufficient number of mission design variables. One such method is NASA’s OpenMDAO optimization tool, an open-source, Python-based framework used “for building new analysis tools with analytic derivatives” [6]. By partitioning the mission into several modules (or components) and defining partial derivatives of the outputs with respect to the inputs, the framework allows mission planners to design trajectories that can minimize a variety of variables. A deeper explanation of the OpenMDAO structure in regards to this project is discussed in Section III.

For this project, a complete Python optimization program is developed using the OpenMDAO framework, in order to study how optimal transfer trajectories from Earth to Mars can be defined and optimized. In particular, the program is used to minimize the amount of delta-V for this mission type, given a specified Earth parking orbit and Mars parking orbit.

This paper places a strong focus on rigorously defining an optimization objective function that serves to model the mission at large. In Section II, the optimization problem is defined and derived, using content from M.R Emami’s Lecture Notes from the course AER1503 and [5]. Section III presents

an overview of the Python program written to optimize the objective function, along with any working assumptions made. Here, a brief introduction to OpenMDAO's inputs, outputs, components, and groups is also provided. Optimization results are detailed in Section IV, along with a discussion on their relevance in Section V. The paper then concludes with a summary of key findings and possibilities for future study in Section VI.

II. DERIVATION OF OPTIMIZATION PROBLEM

This section develops a formalism for the objective function that the OpenMDAO program will eventually optimize. It should be noted that given the current limitations of OpenMDAO's optimization process, the gradient descent method is only able to optimize one objective per optimization. As a result, the objective function (and subsequent program) is developed as an optimization problem seeking to minimize the delta-V (ΔV) used by the spacecraft during the entire interplanetary maneuver.

Throughout this treatment of the problem, Lecture Notes for Chapter 9 from the course AER1503 are referenced [7]. To start, the Earth-Mars mission can be considered to be a two-body problem, for which there are certain perturbative effects. This allows for the spacecraft's motion to be defined as [7]

$$\ddot{\hat{r}} = -\mu \frac{\hat{r}}{r^3} + \hat{p} \quad (1)$$

where μ is the gravitational parameter of the central body, r is the inertial position of the spacecraft, and $\hat{p}$ is the net perturbing acceleration vector that encapsulates perturbations such as "non-spherical gravitational fields, atmospheric drag, solar radiation pressure, gravitational influence of other bodies and propulsive thrust" [7], [8].

Now, since the problem at hand concerns an optimal Hohmann transfer (an impulsive maneuver) from Earth to Mars, the only points of thrust that can be reasonably optimized are the initial burn to enter a heliocentric orbit, and the subsequent secondary burn to enter a Mars capture orbit. While solar radiation pressure, solar gravity and other perturbations continually affect the spacecraft during its flight, their overall effects do not need to be optimized [8]. Therefore, the sole focus of minimizing the total delta-V used is justified.

The mission overall is modeled as a patched conics method, with three distinct phases. In order for the spacecraft to enter a heliocentric transfer orbit from an Earth parking orbit, it must impart a delta-V ΔV_E . Following a period of cruising to Mars, a burn ΔV_M must then be applied to enter a Mars capture (parking) orbit. These two impulses form the bulk of the objective function that is to be optimized. This can be described by the cost function

$$J = \Delta V_E + \Delta V_M \quad (2)$$

However, in order to completely formalize the objective function, perturbative effects can be added into the cost function using methods of variational calculus. Following Poznyak's treatment of the optimal control problem and Chapter 9 Lecture Notes, Equation 2 can be expanded to include perturbative

terms as a Bolza problem [7], [9]. This "cost functional" takes the form

$$J_{\text{bolza}} = J + \int_{t_0}^{t_f} h(x(t), u(t), t) dt \quad (3)$$

where t_0, t_f denote the mission start and end dates respectively, and the Lagrange function h is defined as

$$h(x(t), u(t), t) = \hat{r} + \mu \frac{\hat{r}}{r^3} - \hat{p} \quad (4)$$

The objective function is now updated to be

$$J_{\text{bolza}} = J + \int_{t_0}^{t_f} \hat{\lambda}(t) \left[\hat{r} + \mu \frac{\hat{r}}{r^3} - \hat{p} \right] dt \quad (5)$$

where the Lagrange multiplier $\hat{\lambda}$ develops a penalty term that acts as an optimization constraint (following the Variation of Parameters section from [7]).

Formally, the developed optimization problem is stated as

$$\text{minimize } J_{\text{bolza}} \quad (6)$$

$$\text{subject to } g_i(x), \quad i = 1, 2, \dots, n \quad (7)$$

where $g_i(x)$ are constraints imposed on the system [10].

As it stands, notable constraints accounted for are the Lagrange term of Equation 5, along with restrictions placed as bounds on the radii of Low Earth Orbit (r_{LEO}) and Low Mars Orbit (r_{LMO}). These constitute design variables for the OpenMDAO model [11],

$$6538.0 \text{ km} \leq r_{LEO} \leq 8378.0 \text{ km} \quad (8)$$

$$3490.0 \text{ km} \leq r_{LMO} \leq 3590.0 \text{ km} \quad (9)$$

Additionally, a possible constraint can be about the change in inclination for the spacecraft's trajectory, particularly so that [10]

$$\Delta i_{\text{tot}} = \Delta i_E + \Delta i_M = 1.85^\circ \quad (10)$$

where Δi_E describes the inclination of the spacecraft's orbit at the instance of departure and Δi_M describes the arrival inclination at Mars [5].

III. DEFINITION AND DEVELOPMENT OF EARTH-MARS MISSION USING OPENMDAO

To now consider the optimization of the objective defined in the previous section, the development of a simplified (coplanar) Hohmann transfer mission between LEO and LMO using the OpenMDAO framework must first be detailed. Since the effects of perturbations do not strongly alter the trajectory of the spacecraft (any notable effects are assumed to be handled by the onboard guidance system), the objective function that is implemented into Python code is in fact Equation 2, coupled with the constraints given by Equations 8 and 9. This decision is also made due to the single-objective optimization limitation of OpenMDAO.

The constraint suggested by Equation 10 is feasible for implementation, however because the heliocentric orbits of Earth and Mars are nearly coplanar, a mere 1.85° difference does

not notably affect the optimization. Thus, its implementation is skipped for this particular project.

To model both an orbit change and rendezvous with Mars upon arrival, the method of patched conics is used to consider the Earth departure and Mars capture as separate conic sections, along with the heliocentric orbit as well [12].

Before seguing into a discussion of how the model is developed in Python code, it is important to understand the underlying concepts of how the OpenMDAO framework operates. The following content heavily references the OpenMDAO Theory Manual, along with a Hohmann transfer tutorial on the developers’ website [10], [13]. In addition, Figure 2 provides a visual related to the following discussion of OpenMDAO’s various modules.

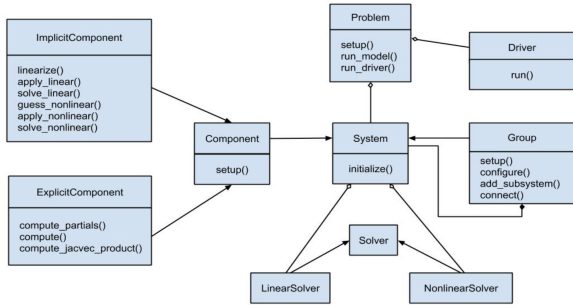


Fig. 2. A web map of how the different parts of the OpenMDAO architecture interact with each other, along with what methods are used within each part. This image is taken from the OpenMDAO Theory Manual on the framework’s website [13].

OpenMDAO is an open-source Python-based framework developed at NASA, for the purposes of systems analysis and multidisciplinary optimization. At a high level, OpenMDAO consists of Systems, which can be either Groups or Components [13]. Groups act as “containers” which can contain Components, other Groups, or a mix. The relationship between parts of a Group can be thought of as a top-down tree, where the highest level structures are Groups and the lowest are Components [13]. This hierarchy allows Groups to effectively manage the dependencies between all of its subsystems, while also supporting the use of linear and nonlinear solvers [13].

On the other hand, Components provide low-level functionality that represent steps such as basic calculations in the overall process [13]. Inputs are passed into Components and passed back out as Outputs, after the Component performs intermediate operations. Outputs can then be passed as Inputs into other Components to support further calculations. It is within these Components that partial derivatives of all of the Outputs with respect to each Input are defined. These derivatives provide gradient information to the optimization solver used at the highest level, enabling quick and robust objective function convergence [13]. There are two types of Components (Implicit and Explicit), but

for the purposes of this optimization problem, this paper only considers ExplicitComponents.

Following the definition and connection of various Groups and Components via Inputs and Outputs, instances of an OpenMDAO Problem and Model are defined, which take in Groups and Components as subsystems [13]. Model constraints and optimization design variables are added at this step, which sets the stage for an OpenMDAO Driver to be run. Running the Driver allows linear or nonlinear Solvers to run, which are responsible for managing the Model’s execution. For this mission type in particular, gradient descent optimization is used. It is also important to note that OpenMDAO automatically performs unit conversions between subsystems, as long as they are provided to the Solvers [13].

Now that some background of how OpenMDAO functions has been provided, the mission’s development can be discussed. The content here largely follows the OpenMDAO Hohmann transfer example, albeit with quite a significant number of differences [10].

First, it is assumed that a spacecraft starts out in Low Earth Orbit, and an OpenMDAO ExplicitComponent (EarthOrbitComp) is created. This component takes in values for r_{LEO} and μ_{Earth} (Earth’s gravitational parameter) as Inputs, and then computes a spacecraft’s needed circular orbital velocity v_{LEO} using the equation [14]

$$v_{LEO} = \sqrt{\frac{\mu_{Earth}}{r_{LEO}}} \quad (11)$$

Partial derivatives for v_{LEO} with respect to r_{LEO} and μ_{Earth} are defined in EarthOrbitComp that are later used to provide gradient information to the solver.

After obtaining v_{LEO} , the HelioTransferOrbitComp and HelioInjectionMatchComp components compute parameters that are required for the spacecraft to be injected into a heliocentric transfer orbit. Firstly, HelioTransferOrbitComp takes as Inputs the heliocentric orbit radii of Earth and Mars ($r_{Earth,h}$ and $r_{Mars,h}$ respectively) and the Sun’s gravitational parameter μ_{Sun} , with which it Outputs the spacecraft’s heliocentric orbit injection speed $v_{dep,h}$, its heliocentric capture speed $v_{arr,h}$ and the transfer ellipse’s time of flight tof_h . Explicitly, these calculations are given by

$$v_{dep,h} = \sqrt{\frac{2\mu_{Sun}}{r_{Earth,h}} - \frac{2\mu_{Sun}}{r_{Earth,h} + r_{Mars,h}}} \quad (12)$$

$$v_{arr,h} = \sqrt{\frac{2\mu_{Sun}}{r_{Mars,h}} - \frac{2\mu_{Sun}}{r_{Earth,h} + r_{Mars,h}}} \quad (13)$$

$$tof_h = \pi \sqrt{\frac{(r_{Earth,h} + r_{Mars,h})^3}{8\mu_{Sun}}} \quad (14)$$

Next, the HelioInjectionMatchComp subsystem component computes the hyperbolic excess speed $v_{\infty,E}$ at Earth required for injection into the transfer trajectory. This

component takes inputs of $v_{\text{dep},h}$, $r_{\text{Earth},h}$ and μ_{Sun} in order to output $v_{\infty,E}$ via the equation [8]

$$v_{\infty,E} = \left| v_{\text{dep},h} - \sqrt{\frac{\mu_{\text{Sun}}}{r_{\text{Earth},h}}} \right| \quad (15)$$

Following the above computations, another subsystem `EarthEscapeDVComp` computes the delta-V ΔV_E required for the spacecraft to depart LEO and enter a heliocentric transfer orbit. This subsystem takes in v_{LEO} and the previously computed value of the required hyperbolic excess speed $v_{\infty,E}$ required to leave LEO. The major calculation performed in this subsystem is done using the formula

$$\Delta V_E = \sqrt{v_{\text{LEO}}^2 + v_{\infty,E}^2} - v_{\text{LEO}} \quad (16)$$

At this point, the spacecraft has entered a heliocentric Hohmann transfer orbit, where it coasts until it is “captured” by Mars’ sphere of influence. In this phase of the mission, there are several OpenMDAO subsystems that contribute to the parameters needed for a successful patched conics interplanetary trajectory.

To begin, the `MarsInjectionMatchComp` component computes the hyperbolic excess speed needed at Mars’ arrival. This component takes $v_{\text{arr},h}$, $r_{\text{Mars},h}$ and μ_{Sun} as inputs, with which it outputs the hyperbolic excess speed $v_{\infty,M}$ at Mars. The calculation in this component is given by [8]

$$v_{\infty,M} = \left| v_{\text{arr},h} - \sqrt{\frac{\mu_{\text{Sun}}}{r_{\text{Mars},h}}} \right| \quad (17)$$

The `MarsOrbitComp` component then determines what the required circular orbital velocity for the spacecraft to maintain Low Mars Orbit is. Taking a specified Mars parking orbit radius r_{LMO} (which is a design variable) and Mars’ gravitational parameter μ_{Mars} as inputs, the component computes the required speed v_{LMO} as [14]

$$v_{\text{LMO}} = \sqrt{\frac{\mu_{\text{Mars}}}{r_{\text{LMO}}}} \quad (18)$$

Finally, the `MarsCaptureDVComp` component calculates what the required delta-V is for Mars orbit insertion. The component takes as inputs $v_{\infty,M}$ and v_{LMO} , outputting the delta-V burn required to enter LMO as

$$\Delta V_M = \sqrt{v_{\text{LMO}}^2 + v_{\infty,M}^2} - v_{\infty,M} \quad (19)$$

The total delta-V needed for this mission is then described by Equation 2, which is what the Python program seeks to optimize.

IV. OPTIMIZATION RESULTS

The work in this section builds upon the midterm progress work presented in [12], with additional depictions of tests and subsequent function convergence rates.

Prior to the optimization, the model is first run to calculate a non-optimal delta-V, using equations described in [8] and [14]. Table I displays the non-optimized results for three different trials, where the design variables are fixed to be at the

beginning, midpoint, and end of their allowed ranges, given by Equations 8 and 9.

TABLE I
TABLE OF NON-OPTIMIZED ΔV RESULTS

Trial	Initial r_{LEO}	Initial r_{LMO}	Total ΔV
1	6538 km	3490 km	2.280 km/s
2	7458 km	3540 km	2.294 km/s
3	8378 km	3590 km	2.306 km/s

Using the `run_driver` method of the OpenMDAO Problem class, the results of the previously run model are then optimized. Now, given that the Hohmann transfer is commonly regarded as “the most energy-efficient approach for a spacecraft to transfer from one planet’s orbit to another”, the optimal delta-V is expected to be close to that of the non-optimal delta-V [14]. Table II shows the optimal r_{LEO} and r_{LMO} corresponding to the trials shown in Table I, along with their respective optimized delta-V values.

TABLE II
TABLE OF OPTIMIZED ΔV RESULTS

Trial	Optimal r_{LEO}	Optimal r_{LMO}	Total ΔV
1	6538 km	3590 km	2.240 km/s
2	6538 km	3590 km	2.241 km/s
3	6538 km	3590 km	2.241 km/s

It should be noted that each trial required a different number of iterations (and subsequent function evaluations) to converge. The results of each trial’s convergence rate (with respect to the objective function value) is displayed in Figure 3.

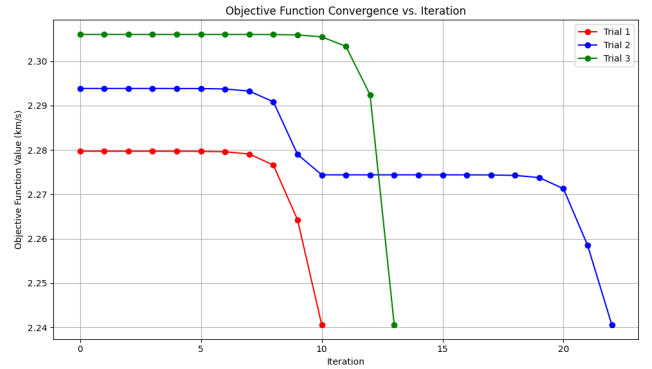


Fig. 3. A plot showing the convergence of the objective function value (Equation 2) versus the number of iterations. The iterations begin from the values shown in Table I, and converge to the values in Table II.

Furthermore, Figure 4 shows the convergence of the design variables (Equations 8 and 9) as a function of the number of iterations.

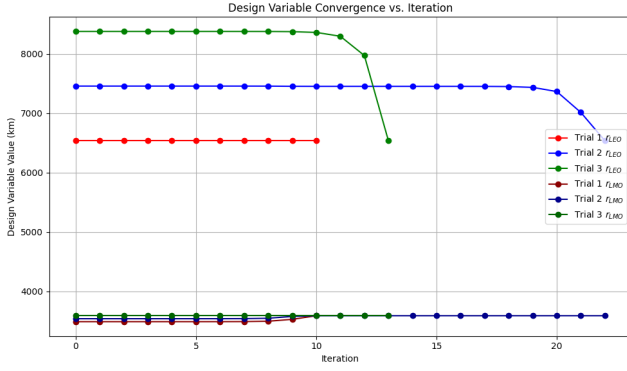


Fig. 4. A plot showing the convergence of the design variable values versus the number of iterations. The iterations begin from the values shown in Table I, and converge to the values in Table II.

An analysis and discussion of the results above is presented in the next section.

V. DISCUSSION

Running the Python optimization on the problem defined in Section III produces highly interpretable results.

Immediately, it is clear that the OpenMDAO program produces optimized delta-V values that are very close to the non-optimized values. According to Table I, the total non-optimized delta-V values range from 2.280 – 2.306 km/s. These values are calculated using the orbital mechanics equations available in literature for interplanetary travel using the method of patched conics, which does not necessarily give the most optimal results. However, the data from Table II shows that the optimized delta-V values range from 2.240 – 2.241 km/s. These values are only about 1.7 – 2.9% smaller than the non-optimized values, implying that the gradient descent algorithm did not deliver a significant reduction in delta-V. This result can however be explained by the fact that the Hohmann transfer is already the optimal transfer maneuver, given that it is a transfer between coplanar planets in circular heliocentric orbits [14]. In other words, this result provides a supporting argument which supports the validity of the optimization program [14].

In addition, analysis of Figures 3 and 4 shows that the `ScipyOptimizeDriver` converges in less than 25 iterations for all three initial guesses. This robust convergence supports the importance of providing as many partial derivatives as possible to the OpenMDAO framework, as opposed to simply letting the tool compute derivatives on its own [13]. Furthermore, both Figure 3 and 4 show that the optimizer converges to the same values for the optimized delta-V, r_{LEO} and r_{LMO} . This can be attributed to the programmed model’s relative simplicity, since perturbative effects are not considered. In practice, effects such as atmospheric drag would alter the bounds set on r_{LEO} and r_{LMO} , which would go beyond what is listed in this paper.

VI. CONCLUSIONS AND FUTURE WORK

For this paper, the development of an end-to-end Python interplanetary trajectory optimization program is detailed, us-

ing concepts from the OpenMDAO framework and various sources of literature. The task at hand concerns the trajectory optimization of a classical Earth-Mars Hohmann transfer, a problem outlined extensively in [5] and [14]. Specifically, the delta-V minimizing two-burn problem is recreated in Python code using the method of patched-conics and the modular optimization programming structure described by [13].

The optimization’s objective function is rigorously defined in Section II, while referencing [8], [9] and [14]. Particularly, the theory of the calculus of variations is used to reconcile perturbative effects a spacecraft might experience during flight with the primary objective of minimizing total delta-V, in order to formulate a generalized cost functional J_{bolza} (Equation 5). Due to the limitations of OpenMDAO however, the working objective function is simplified to involve only the sum of heliocentric injection and Mars orbit capture burns, given by Equation 2.

A brief introduction to OpenMDAO and its methodology is given in Section III. Here, the various parts of the mission are defined in terms of OpenMDAO’s class structure, with descriptions of how mathematical equations are used provided for each step.

Results of the optimization on a test suite of initial design variable values are shown in Section IV, along with an analysis of their significance in Section V. It was found that the non-optimized delta-V values for each trial were notably close to the actual optimized values produced by the OpenMDAO optimizer; it was concluded that since the Hohmann transfer is already the most optimal interplanetary maneuver for coplanar orbits, there is not much room for improvement.

Several paths can now be taken in order to extend the framework and current program. Incorporating the 1.85° plane-change constraint and additional considerations such as launch-window opportunities can lead to more significant delta-V results. Adding the current working model into a larger and more comprehensive multi-objective optimizer can help reconcile the trade-off between delta-V, time-of-flight and spacecraft mass, potentially leading to full round-trip crewed-mission studies. Under these considerations, the present prototype can develop into a versatile mission-design code capable of exploring more complex ideas, such as non-Hohmann transfers. Ultimately, the current program and methodology sets a strong foundation for future studies on interplanetary trajectory optimization.

REFERENCES

- [1] R. Kothari, “AER1503 project definition,” University of Toronto Institute of Aerospace Studies, Toronto, ON, Canada, Tech. Rep., 2025.
- [2] D. Williams. (2011) Chronology of mars exploration. NASA Goddard Space Flight Center. Accessed: 2025-03-01. [Online]. Available: https://nssdc.gsfc.nasa.gov/planetary/chronology_mars.html
- [3] B. Weber. (2021, Oct.) Hohmann transfer. Accessed: 2025-04-05. [Online]. Available: <https://orbital-mechanics.space/orbital-maneuvers/hohmann-transfer.html>
- [4] W. Hohmann, *The Attainability of Heavenly Bodies*. Washington: NASA, 1960.
- [5] H. D. Curtis, *Orbital Mechanics for Engineering Students*, 4th ed., ser. Elsevier Aerospace Engineering Series. San Diego, CA, USA: Elsevier, 2019.

- [6] J. S. Gray, J. T. Hwang, J. R. R. A. Martins, K. T. Moore, and B. A. Naylor, "OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization," *Structural and Multidisciplinary Optimization*, vol. 59, no. 4, pp. 1075–1104, April 2019.
- [7] M. Emami, "Chapter 9 lecture notes for AER1503," 2025, lecture notes, University of Toronto.
- [8] V. U. J. Nwankwo and S. K. Chakrabarti, "Analysis of planetary and solar-induced perturbations on trans-martian trajectory of mars missions before and after mars orbit insertion," *Indian Journal of Physics*, vol. 89, no. 12, p. 1235–1245, May 2015. [Online]. Available: <http://dx.doi.org/10.1007/s12648-015-0705-9>
- [9] A. S. Poznyak, *Variational Calculus and Optimal Control*. Elsevier, 2008, p. 647–711. [Online]. Available: <http://dx.doi.org/10.1016/B978-008044674-5.50025-0>
- [10] OpenMDAO Development Team. (2022) Hohmann Transfer Example – Optimizing a Spacecraft Maneuver. NASA. Accessed: 2025-03-01. [Online]. Available: https://openmdao.org/newdocs/versions/latest/examples/hohmann_transfer/hohmann_transfer.html
- [11] Center for Strategic and International Studies. (2017) Popular orbits 101 – aerospace security. Accessed: 2025-03-01. [Online]. Available: <https://aerospace.csis.org/aerospace101/earth-orbit-101/>
- [12] R. Kothari, "AER1503 project progress report," University of Toronto Institute of Aerospace Studies, Toronto, ON, Canada, Tech. Rep., 2025.
- [13] OpenMDAO Development Team. (2022) OpenMDAO Theory Manual. NASA. Accessed: 2025-04-05. [Online]. Available: https://openmdao.org/newdocs/versions/latest/theory_manual/theory_manual.html
- [14] M. Emami, "Chapter 10 lecture notes for AER1503," 2025, lecture notes, University of Toronto.